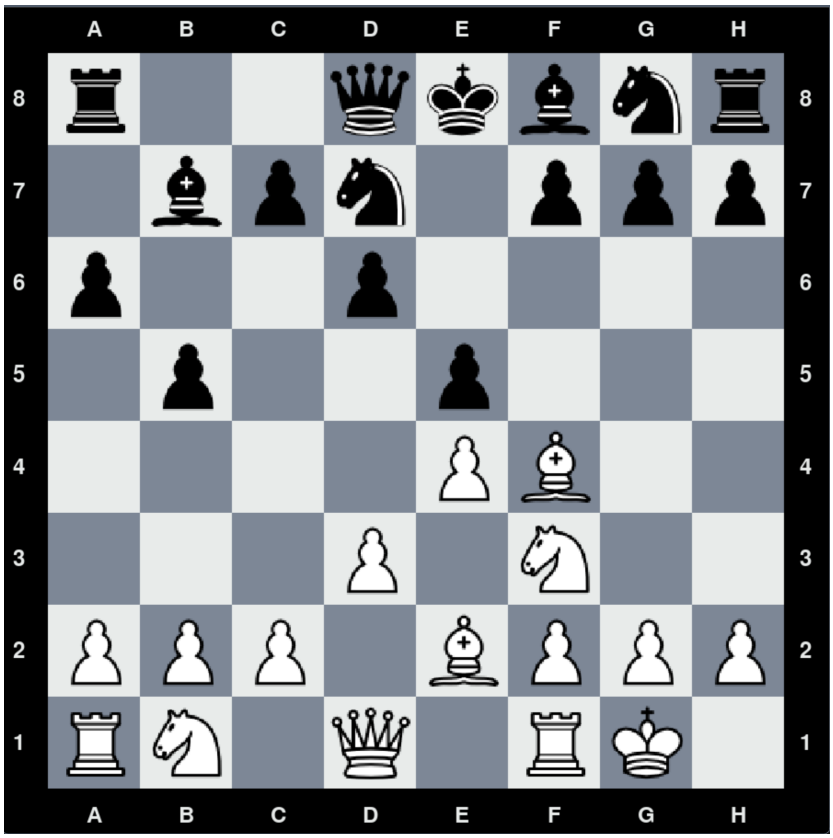
Kantonsschule Im Lee Winterthur

Maturitätsarbeit HS 2024/25



Designing Chess Playing Programs using Deep Learning Algorithms

Banuijan Yognanathan 4a, Supervising Teacher : Cyril Wendl

Winterthur, January 6th 2025

Contents

0 Introduction **3**
0.1 *Idea and Motivation* . . . 3
0.2 *Neural Network and Deep Learning* . . . 3

1 Methodology **4**
1.1 *Choosing the right Network* . . . 4
1.2 *Preprocessing the Data* . . . 5
1.2.1 *Feature Maps* . . . 6
1.2.2 *Represent a Move* . . . 7
1.2.3 *Prepare Data from the Raw Dataset* . . . 9
1.2.4 *Chess Move Notation* . . . 11
1.3 *Training Method* . . . 11
1.3.1 *Training Mechanism* . . . 11
1.3.2 *Structure of the CNN* . . . 13
1.3.3 *The Data Loader* . . . 16
1.3.4 *Training Loop* . . . 17
1.3.5 *Losses* . . . 20
1.4 *Creating the GUI* . . . 20

2 Results **21**
2.1 *Performance of the Neural Network* . . . 21
2.2 *Designing the Graphical User Interface* . . . 21

3 Discussion **24**
3.1 *Improving the AI* . . . 24
3.2 *GUI* . . . 24

4 Conclusion **25**

A Appendices **26**
A.1 Preprocessing Script 26
A.2 CNN Configuration Script 28
A.3 My Personal GitHub 31

0 Introduction

0.1 *Idea and Motivation*

Artificial Intelligence (AI) is prevalent in the world today. Undoubtedly, its numerous areas of application like *"science, engineering, business, medicine or weather forecasting"* [1] and the recent uprise of specific forms of AI such as ChatGPT justify its popularity. Significant fields where AI verifies to have high functionality are *Expert Systems, Natural Language Processing, Speech Understanding, Robotics* and many more [1]. As an aspiring programmer and passionate chess player, merging these two subjects was of great interest. Particularly when viewing existing chess engines like Stockfish or Maia Chess. Therefore, this project is about **designing chess-playing systems using Deep Learning (DL) techniques**.

0.2 *Neural Network and Deep Learning*

Deep Learning is the first building block of any Artificial Intelligence, as illustrated in Figure 1 . A fundamental structure used for DL and other related processes is a Neural Network (NN). A classic Neural Network consists of multiple simple, connected processors called **Neurons** (x), which get activated through weighted connections from the

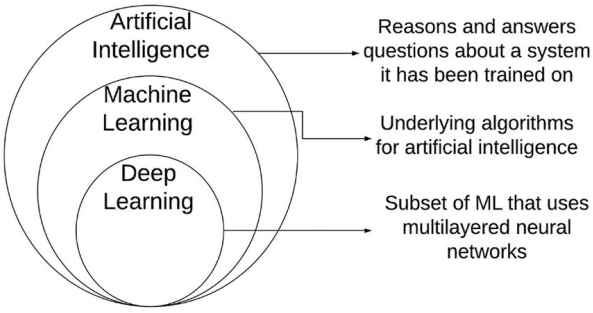


Figure 1: Hierarchy of AI subfields [2]

previously activated Neurons [3]. The connection between two Neurons is called a **Weight** (w), whereas the value a Neuron gets activated by is referred to as an **Activation** (a). The total Activation number of a Neuron is calculated by multiplying all the Weights and Activations from the previous layer [4], as shown in the following formula :

$$a_i = \phi\left(\sum_{j=1}^{n} w_{ij}x_j + b_i\right)$$

Additionally, a **Bias** (b is added. This way, only the meaningful neurons with a high number are computed for the layer [4]. An **Activation Function** (ϕ) is used to add non-linearity, as we will see later in Section 1.3.2. Learning for a computational algorithm is about finding weights that make the NN behave in a desired way [3]. For this very project, this means that a Neural Network has to be designed in such a way that the NN can play a chess game without rule violation. Deep Learning includes finding an appropriate and accurate composition of the earlier discussed components over many iterations [1].

1 Methodology

1.1 *Choosing the right Network*

For the purpose of designing a chess-playing engine based on ML algorithms, there seem to be several options. The type of NN which will be used for this very project is a **Convolutional Neural Network (CNN)**. This kind of network has shown excellent performance in many computer vision and machine learning

problems [5]. The functionality of this type of neural network relates to how the network should be trained, as "CNNs are useful in various applications, especially in image-related tasks" [5]. CNN applications include *image classification, image semantic segmentation, object detection in images*, and many more [5].

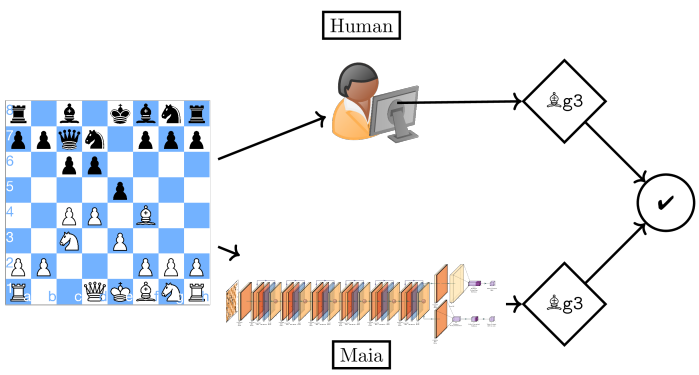


Figure 2: Human vs. AI chess strategy [6]

The idea of training a chess model based on image pattern recognition sounds very intuitive. Compared to a human being, the AI should choose the best move based on the numerous chess board states which it will be trained with, shown in Figure 2. Therefore, it chooses his move by recognizing spatial features and patterns, which appears to be the strength of CNNs. The **intuitiveness** of this type of network is one of the main reasons for choosing a CNN as model. The use of unique board states as input to an NN promises good rewards for a chess system.

1.2 *Preprocessing the Data*

Data processing is typically the very first phase of any programming project related to constructing an NN. One makes sure that the input data is prepared such that it is **conducive to learning**, allowing the CNN to extract spatial features and patterns.

1.2.1 *Feature Maps*

As shown in Figure 3, so-called **Feature Maps**, which are nothing else but 2-D or 3-D arrays, will act as **input** to the NN. For every single type of chess piece, such a feature map is created. The matrix representation in the Figure fits well with the input type of a CNN, since " tensors of order 3 or higher are frequently used in CNNs" [5].

```

array([[[ 0,  0,  0,  0,  0,  0,  0, 0],
    [-1, -1, -1, -1, -1, -1, -1, -1],
    [ 0,  0,  0,  0,  0,  0,  0,  0],
    [ 0,  0,  0,  0,  0,  0,  0,  0],
    [ 0,  0,  0,  0,  0,  0,  0,  0],
    [ 0,  0,  0,  0,  0,  0,  0,  0],
    [ 1,  1,  1,  1,  1,  1,  1,  1],
    [ 0,  0,  0,  0,  0,  0,  0,  0]]],

```

Figure 3: Feature Map of Pawns

The idea is to generate feature maps for each of the six types of chess pieces. The number 1 symbolizes the white chess pieces, while all of the positions labeled with the number -1 represent the black pieces. To create such matrix representations, the well-documented Python Chess module was utilized for this project. This module is useful because it delivers many **built-in functions** which have been confirmed to be useful. One of them recalls a representation of the chess board as seen in Figure 4.

```

r n b q k b n r
p p p p p p p p
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
P P P P P P P P
R N B Q K B N R

```

Figure 4: Chess Board Representation from Python Chess Module

Based on this template, one only has to substitute the desired type of chess piece, which is characterized by a unique letter, with the corresponding number, namely 1 or -1. Finally, one converts this board representation to an NumPy array. The **create_rep_layer** function, located in the **Preprocessing Script**, creates feature maps for every single type of chess piece.

1.2.2 *Represent a Move*

The feature maps, discussed in Section 1.2.1, will aid us in representing a chess move. Again, the idea is to come up with matrix representations. In this case, two different matrices, namely the **from-matrix** for the moving piece at the beginning, and the **to-matrix**, for the piece after the move has been played, are constructed. In Figure 5, the concept for the move "e2e4" is shown.

```
#          "from matrix"                    "to matrix"

# 8     [[0 0 0 0 0 0 0 0]        8     [[0 0 0 0 0 0 0 0]
# 7     [0 0 0 0 0 0 0 0]         7     [0 0 0 0 0 0 0 0]
# 6     [0 0 0 0 0 0 0 0]         6     [0 0 0 0 0 0 0 0]
# 5     [0 0 0 0 0 0 0 0]         5     [0 0 0 0 0 0 0 0]
# 4     [0 0 0 0 0 0 0 0]         4     [0 0 0 0 1 0 0 0]
# 3     [0 0 0 0 0 0 0 0]         3     [0 0 0 0 0 0 0 0]
# 2     [0 0 0 0 1 0 0 0]         2     [0 0 0 0 0 0 0 0]
# 1     [0 0 0 0 0 0 0 0]]        1     [0 0 0 0 0 0 0 0]]

#         A B C D E F G H                 A B C D E F G H
```

Figure 5: Basic Idea for Move Representation

A high-level function called **move_2_rep** that creates those matrices seen in Figure 5 can be found in the **Preprocessing Script**. One only has to determine the row and the column of the chess piece in the from-matrix, as well as in the to-matrix. Knowing the **positions**, their number is converted respectively. This approach seems very intuitive and is a suitable way for the Neural Network to learn. With this method, the NN learns how the individual pieces move along the board, simultaneously learning how to play the game of chess. The realization of the concept is portrayed in Figure 6.

```

array([[[0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 1., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.]],

        [[0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 1., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.]]])

```

Figure 6: Move Representation

1.2.3 *Prepare Data from the Raw Dataset*

After discussing how the chess board can be represented in matrices in a way that is conducive to learning, we will now concentrate on how the **raw data** has to be prepared, such that it matches with the earlier created concepts. A file with Comma-seperated values (CSV), containing **over six million chess**

games found on Kaggle is used as the dataset. There is information in this dataset which is intentionally cut out since it's not relevant to this project. The three columns seen in Figure 7 will play a pertinent role in the preparation of the dataset. **Elo** is a measurement of the skill level of a player. As the skill level of the AI should evolve fairly high, this measurement will be crucial for the overall performance of the game. The higher one's elo, the better he is at chess. The far right column (**AN**) contains **all chess moves that have been played**. This dataset is rich in information since it holds plenty of games and therefore numerous chess moves for our CNN to learn from. After clearing out games which seemed to be too short and inappropriate for the NN to train with, as well as filtering out games with an elo lower than 2000, approximately **900'000 chess games** were

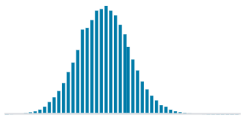
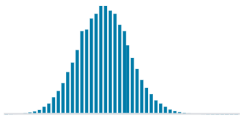
# WhiteElo White's ELO	# BlackElo Black's ELO	AN Movements in Movetext format.
 737 3110	 728 3108	**6188849** unique values
1641	1627	1. e4 e5 2. b3 Nf6 3. Bb2 Nc6 4. Nf3 d6 5. d3 g6 6. Nbd2 Bg7 7. g3 Be6 8. Bg2 Qd7 9. O-O O-O 10. c3 ...
1647	1688	1. e4 d5 2. exd5 Qxd5 3. Nf3 Bg4 4. Be2 Nf6 5. Nc3 Qd8 6. O-O Nc6 7. Bc4 e6 8. h3 Bh5 9. d3 Bd6 10. ...
1706	1317	1. e3 Nf6 2. Bc4 d6 3. e4 e6 4. Nf3 Nxe4 5. Nd4 Nxf2 6. Kxf2 Qh4+ 7. g3 Qxg3+ 8. Kxg3 d5 9. Bb5+ c6 ...

Figure 7: The Raw Data from the Dataset

still available to use, which surely is more than enough for this project.

```

print(chess_data.shape[0])
883376

```

1.2.4 *Chess Move Notation*

The use of feature maps was introduced in Section 1.2.1. The functions creating them require a specific form of input. However, the notation in the dataset seen in Figure 7 is different from the requested notation. The raw dataset provides the game in the so-called **Standard Algebraic Notation (SAN)**. The purpose of the **create_-move_list function** found in the **Preprocessing Script** is to convert the notation of the chess moves in the raw dataset more conveniently, such that the creation of the feature maps is achievable.

1.3 *Training Method*

The Preprocessing finally reached an end. The CNN can work with the input, namely the **feature maps**. Now, it's time to deal with the Neural Network itself. Many aspects can be discussed regarding the NN and one could go more in detail. However, this paper will focus on the relevant elements.

1.3.1 *Training Mechanism*

Take a look at one of the most important functions concerning the training method:

```python

def __getitem__(self, index):
    #random integer from "Dataset"
    game_i = np.random.randint(self.games.shape[0])
    random_game = chess_data['AN'].values[game_i]
    #clean up notation
    moves = create_move_list(random_game)
    #pick random move
    game_state_i = np.random.randint(len(moves)-1)
    next_move = moves[game_state_i]
    #all moves until current move
    moves = moves[:game_state_i]
    board = chess.Board()
    #loop through all moves
    for move in moves :
        board.push_san(move)
    #feature map for board state
    x = board_2_rep(board)
    #represent move in "from" and "to" matrix
    y = move_2_rep(next_move, board)
    if game_state_i % 2 == 1:
        x*= -1
    #return created feature maps
    return x, y

```

Figure 8: The Code for the Training Mechanism

This function describes the **training mechanism** of the Neural Network. Using the Random module, one picks a random game and a **random game state**, as well as

the **next move**. In this randomly chosen game, every individual move which has been played before the randomly chosen game state is **looped through**. The board is updated using the **push_san()** function. Next, **feature maps** are created for the current board state, as well as for the next move. If the index of the game state is uneven, x is multiplied by -1, meaning it is black's turn.

1.3.2 *Structure of the CNN*

In Section 1.1, the purpose of choosing a **Convolutional Neural Network** and its **benefits** have been discussed. In this Section, we will focus on its **structure and functionality**. The source code for the structure of the CNN can be found in the **CNN Configuration Script**. A CNN usually takes an order 3 tensor as its input, for example, an image with X rows, Y columns, and 3 channels, namely the RGB colour channels [5]. This applies perfectly with the feature maps, discussed in Section 1.2.1. As shown in Figure 3, it truly is a **3-D array of 2-D feature maps**, each consisting of **8 rows and 8 columns**. One can assume that there are **6 channels** for every chess piece type. Besides possessing an **Input Layer** as well as an **Output Layer**, what differentiates a **Simple Neural Network** from a **Deep**

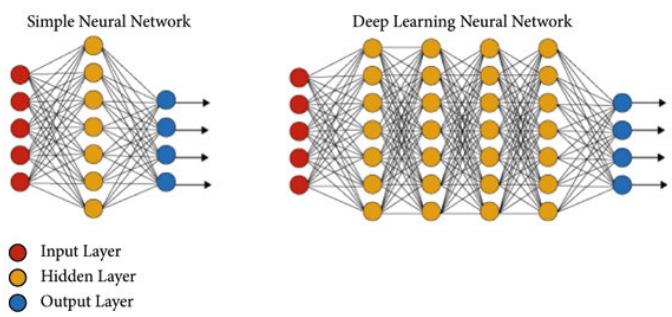


Figure 9: Structure of a Simple NN vs CNN [7]

Neural Network is the number of **Hidden Layers** [8], as seen in Figure 9. Classical Machine Learning (ML) requires only **one Hidden Layer** with the use of **labeled data**, whereas DL usually uses **multiple Hidden Layers**, with the ability to process **unlabeled Data** [8]. ML therefore builds upon DL, shown in Figure 1 [8]. A certain type of layer which can be found in the hidden layers of CNNs is a **Convolutional Layer**. It's where the magic of a CNN comes to appearance, with so-called **Filters** or **Kernels**. In a convolution layer, multiple convolution kernels are usually used [5].

In the network of this project, a **3x3 kernel** that contains specific numbers slides around **8x8 feature maps**, moving 1 pixel at a time for maximal overlap and detail preservation. Those specific numbers, which at the end represent a specific shape or pattern, are determined from the CNN alone during the training process. If the convolution kernel overlaps on top of the input tensor, we can compute the product between the numbers at the same location in the kernel and the input, therefore getting a single number by summing these products together [5]. The calculation is shown in Figure 10. **The higher the number of the product, the higher the probability of obtaining the correct or best output**. Those kernels get bigger and more complex as the input moves on to the following layer. Like

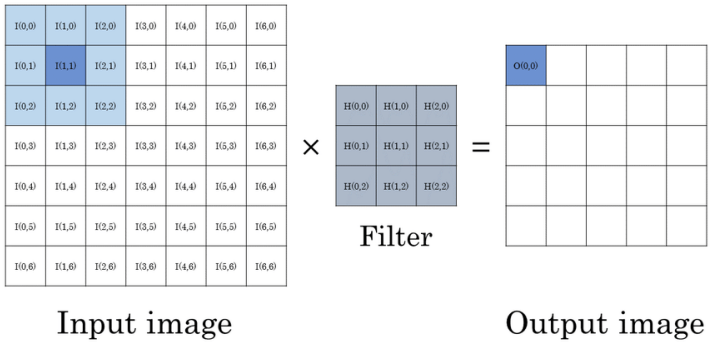


Figure 10: Functionality of a Kernel [9]

this, kernels can determine **specific shapes or edges** in an image. In our case, the CNN can recognize **chess board patterns** on its own, therefore knowing how to play a good move by looking at the state of the board. In between convolutional layers, like in Figure 11, there are so-called **Batch Normalization (BN) layers**. Since the different components of the NN, discussed in Section 0.2, are constantly changed when the input passes through all of the layers, learning can get quite complicated for the CNN. After a while, the training slows down since the learning rate doesn't need to be as high, making it hard to train models with saturating nonlinearities [10]. We make use of BN layers and a phenomenon which is referred to as **internal covariate shift**. This solves the issue "by normalizing layer inputs [10]. This method is well-known and proven to get great results. **Activation Layers** apply a certain transformation to the output of a model, as discussed in Section 0.2. In this project, the Deep Neural Network incorporates neurons with nonlinear activation functions, such as Rectified Linear Unit (RELU) or Scaled Exponential Linear Unit (SELU). This way, the network trains several times faster

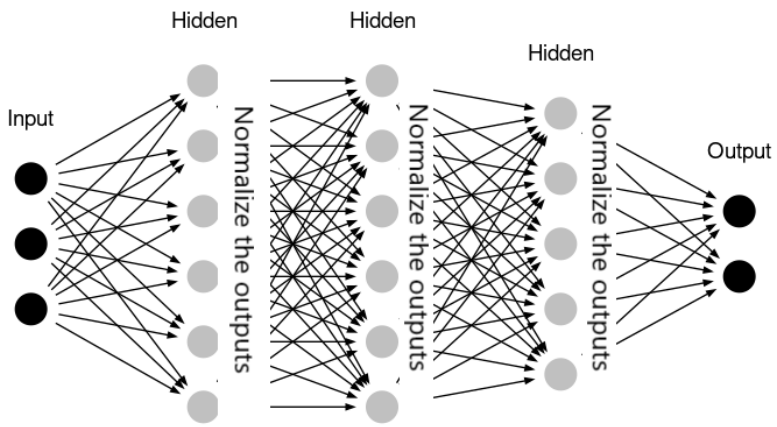


Figure 11: Batch-Normalization Layers [11]

[12]. The Deep Convolutional Neural Network of this very project is composed of **multiple modules** stacked sequentially to process and learn features from the feature maps. To be more precise, it consists of two **Classes**, namely the **Module** and the **ChessNet** Class. The first class is composed of **two convolutional layers**, extracting local spatial patterns and features from the input. Each such layer is followed by a **BN layer** to normalize activations and speed up training. The BN layer is followed by an **activation layer**. Before returning the processed input, the initial input is added on top as this has proven to improve training. This phenomenon is referred to as a **residual connection**. The ChessNet Class is the **overarching network structure**. After receiving 200 feature maps as input that pass through a convolutional layer, this module recalls the previously discussed class iterating through the 4 layers, each processing the output of the ChessNet class. After passing all of the hidden layers, a final convolutional layer will serve as an output layer, by producing a 2 channel output. In other words, we end up with 2 feature maps.

1.3.3 *The Data Loader*

```

    data_train = ChessDataset(chess_data['AN'])
    data_train_loader = DataLoader(data_train, batch_size = 32,
    shuffle = True, drop_last=True)

```

Figure 12: Data Loader

Using the Dataset and the Dataloader module, one can fill the data from the Dataset into **Batches**. 40000 random chess games are chosen to be in the training mechanism discussed in Section 1.3.1. The data loader consists of **32 Batches**. Therefore, every Batch contains 1250 games.

1.3.4 *Training Loop*

After the data has been prepared and the Deep Neural Network has been built, we will now try to understand the training loop. In this Section, we will focus on the main aspects of the training loop, step by step.

```

    # Define the optimizer
    optimizer = optim.Adam(model.parameters(), lr=0.001)

```

With the aid of the PyTorch Module, the so-called **optimizer** is defined. An optimizer adjusts the model's parameters to **minimize the loss function**. The model parameters are all the layers which the input is finally going through. One can recall this function to see, in which layers the input is going through.

```

    num_epochs = 1000
    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0

```

An **Epoch** describes one run through all layers in the network. If the number of epochs is set to 1000, the training data will go through the NN 1000 times respectively. For every epoch, the model, which is the structure of the Neural Network fed with the processed data, is set to training mode. A running loss variable is created to track the loss during training.

```

for i, (x, y) in enumerate(data_train_loader):
    x, y = x.float().to('cuda'), y.to('cuda')
    optimizer.zero_grad()
    output = model(x)
    loss_from = metric_from(output[:, 0, :], y[:, 0])
    loss_to = metric_to(output[:, 1, :], y[:, 1])
    loss = loss_from + loss_to
    loss.backward()
    optimizer.step()
    running_loss += loss.item()

```

In every Epoch, the training data, discussed in Section 1.3.3, is looped through. While x represents the input tensor, meaning the current board state, y represents the target labels, meaning the correct next move. The error of the from- and to-matrix is calculated and appended to the running loss. A process called **Back-propagation** computes a new **gradient** with respect to the loss, whereas the optimizer finally adjusts the weights and biases. Finally, after every epoch, the NN tries to beat itself in a game with 100 moves. The output of the NN is depicted in Figure 13

```

Epoch [15/100], Step [100/1250], Loss: 0.1376
Epoch [15/100], Step [200/1250], Loss: 0.1338
Epoch [15/100], Step [300/1250], Loss: 0.1341
Epoch [15/100], Step [400/1250], Loss: 0.1313
Epoch [15/100], Step [500/1250], Loss: 0.1324
Epoch [15/100], Step [600/1250], Loss: 0.1328
Epoch [15/100], Step [700/1250], Loss: 0.1365
Epoch [15/100], Step [800/1250], Loss: 0.1362
Epoch [15/100], Step [900/1250], Loss: 0.1303
Epoch [15/100], Step [1000/1250], Loss: 0.1339
Epoch [15/100], Step [1100/1250], Loss: 0.1347
Epoch [15/100], Step [1200/1250], Loss: 0.1351
Evaluating model after epoch 15
Game over after 47 moves. Result: 0-1
r . q . . . k .
. p p b . p p p
p . . p . b . .
. . . . . . . n
. . P . . . . .
. . N . . P B .
P P . . . P P P

```

Figure 13: The Output of the Network after an Epoch

1.3.5 *Losses*

```

    # Loss calculation
    metric_from = nn.CrossEntropyLoss()
    metric_to = nn.CrossEntropyLoss()

    # Calculating the loss for "from" and "to" positions
    loss_from = metric_from(output[:,0,:], y[:,0,:])
    loss_to = metric_to(output[:,1,:], y[:,1,:])
    loss = loss_from + loss_to

```

Figure 14: Calculation of the Losses for both Matrices

Using the built-in **CrossEntropyLoss()** function from the torch module, which measures the difference between the model's predicted class probabilities and the true class labels, one only has to calculate the loss of the "from" and "to" positions. The loss is calculated by comparing the predicted "from" and "to" positions.

1.4 *Creating the GUI*

After the NN had been trained, it was time to create a visualization of the chess game on a Graphical User Interface (GUI). This included drawing a board, coming up with a mechanism such that the User is able to engage with the program, and finally, implementing our NN. The GUI has been created using Pygame module. The source code for the GUI can be found on My Personal GitHub.

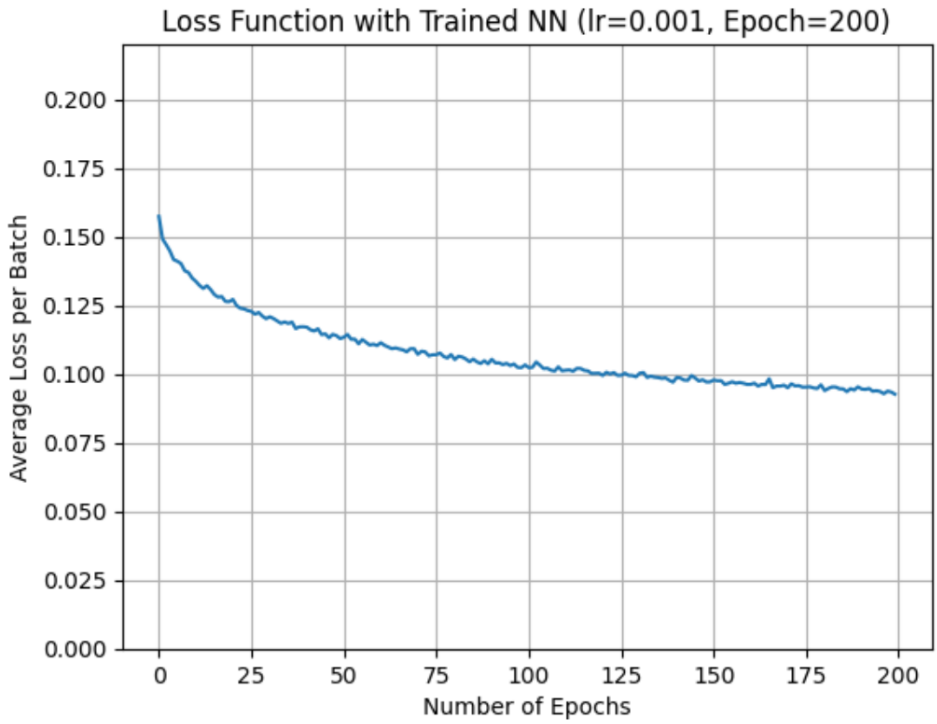
2 Results

2.1 *Performance of the Neural Network*

Figure 16 shows that the loss function decreased with time, which is a sign that the CNN was indeed able to learn from the input data. However, after a certain point, the NN seems to lose the capability to extract features from the input, as the loss function shows swivelling as seen in Figures 15 and 16. Nevertheless, the loss seems to decrease overall. The model seems to give the best results if one trains it either with a **high learning rate and many epochs** or by training the model with a **low learning rate and few epochs**. Hence, just because the loss function is lower, it doesn't mean that the performance of the AI is better, as the swiveling indicates the incapability of the Network to learn from the input data. By simulating a chess game against the AI itself using a high-level function, one can surely recognize sophisticated checkmates from the AI, seen in Figure 15.

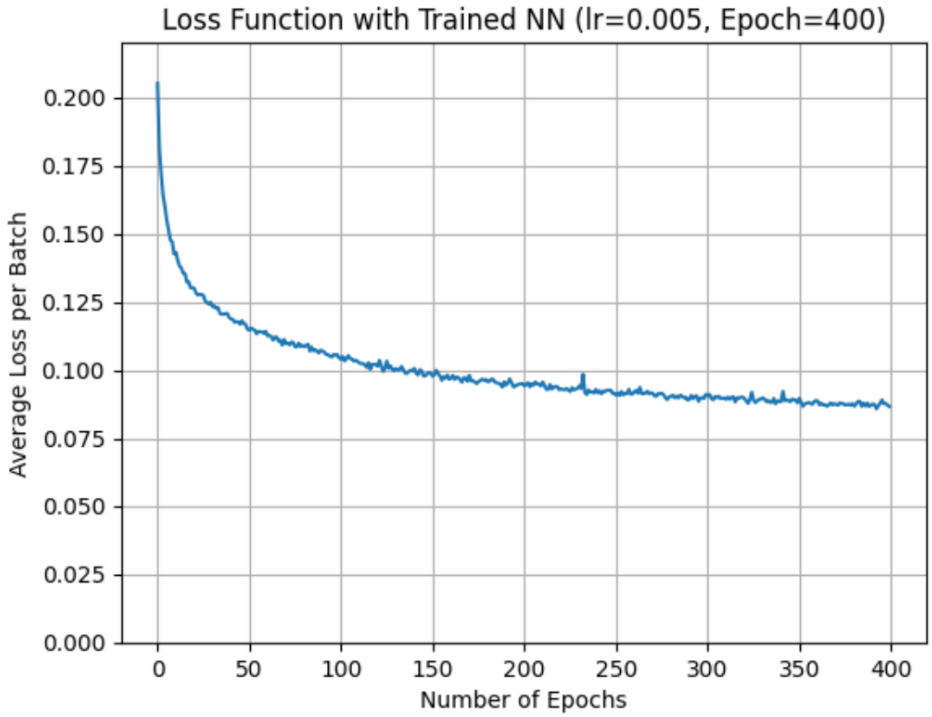
2.2 *Designing the Graphical User Interface*

The GUI has been created using the Pygame module. It allows the Programmer to engage with the User. In this case, a click of the User on the Interface can be interpreted by the program. Not only one is able to design a chess board, but one is also able to handle the clicks of the User. The User Interface is built such that the User can play a chess game against the AI of choice. A snapshot of the chess board is actually seen on the titlepage.




```
Evaluating model after epoch 197
Game over after 78 moves. Result: 1-0
. . . R . . . k
p . . . . . p p
. . . . . . . q
. . . . . . . .
. . . . . . . .
. . . . . P . .
P P . . . B P P
. . . . . . K .
```

Figure 15: Evaluation of a trained NN (Learning Rate = 0.01, 200 Epochs)




```
Evaluating model after epoch 191
Game over after 50 moves. Result: 1-0
. . . . . b . r
p . . . p . p p
. p p . b . p .
. . k . r . . .
. Q . . . . . .
. . P . P B . .
P P . . . P P P
R . . . . R K .
```

Figure 16: Evaluation of a Trained NN (Learning Rate = 0.005, 400 Epochs)

3 Discussion

3.1 *Improving the AI*

Overall, designing chess-playing algorithms using Deep Learning (CNNs to be precise) was successful. Not only did the AI indeed learn from the processed data, but furthermore, the AI was able to play chess in a sophisticated way. Personally, I am satisfied with the results since the main goal of this project has been achieved. If time wasn't limited, I'd focus on improving the NN with other types of learning mechanisms, such as Reinforcement Learning. The AI is definitely not able to compete against advanced chess players, as it still makes basic mistakes. Learning a chess game based on chess position shows flaws, since chess is also about predicting the next moves of the opponent, which would have been an alternative approach. However, I'd assume that learning with a CNN delivers a good basis for the algorithm.

3.2 *GUI*

Creating the GUI consumed more time than initially planned. The initial plan was to improve the AI after creating a simple User Interface. However, creating the GUI caused many problems, especially regarding the implementation of the AI. Therefore, I am pleased with the fact that the NN is able to make decent and valid moves, such that the average chess player can indeed play a solid match against the AI.

4 Conclusion

Although not everything that was planned has been achieved, I am pleased with the fact that I managed to design a chess-playing system using Deep Learning, with a Graphical User Interface for the User to play a game against the AI. This project gave me a new insight into the topic of Artificial Intelligence. I am sure that with further training and developments, the AI is soon able to compete against advanced players. With my new understanding of deep learning and neural networks, the image I had of AI at the time has changed. It's amazing to see what concepts are hidden behind these big names like ChatGPT and the like. Above all, the question of how to deal with artificial intelligence has been raised, as it also appeals to the idea that AI will develop over time in such a way that it will outperform humans, perhaps even take over humanity. Let's see what the future brings.

A Appendices

A.1 Preprocessing Script

```
import re
import gc
import numpy as np
import pandas as pd
import chess
#column index mapping - change letters into numbers and vice
   versa
letter_2_num = {'a':0, 'b':1, 'c':2, 'd':3, 'e':4, 'f':5,
   'g':6, 'h':7}
num_2_letter = {0:'a', 1:'b', 2:'c', 3:'d', 4:'e', 5:'f',
   6:'g', 7:'h'}
# Idea : CNN's can acces 3D inputs !
# Create feature map for each chess piece type {pawn, knight,
   bishop} , white pieces =1, black pieces =-1
# CNN board can learn the rule and make moves based on the
   state of the game
board = chess.Board()

def board_2_rep(board):
  pieces = ['p', 'r', 'n', 'b', 'q', 'k']
  layers = []
  for piece in pieces:
    #create feature map for each chess type
    layers.append(create_rep_layer(board, piece))
  board_rep = np.stack(layers)
  return board_rep

def create_rep_layer(board, type):
```

```
  #replace components
  s = str(board)
  s = re.sub(f'[^{type}{type.upper()} \n]', '.', s)
  s = re.sub(f'{type}', '-1', s)
  s = re.sub(f'{type.upper()}','1',s)
  s = re.sub(f'\.', '0', s)
  board_mat = []
  for row in s.split('\n'):
    row = row.split(' ')
    row = [int(x) for x in row]
    board_mat.append(row)
  return np.array(board_mat)

def move_2_rep(move, board ):
  #convert dataset from SAN into UCI  format : 4 letters {d4e5}
  board.push_san(move).uci()
  move = str(board.pop())
  #determine pos of "from"
  from_output_layer = np.zeros((8,8))
  from_row = 8 - int(move[1])
  from_column = letter_2_num[move[0]]
  from_output_layer[from_row, from_column] = 1
  #determine pos of "to"
  to_ouptut_layer = np.zeros((8,8))
  to_row = 8 - int(move[3])
  to_column = letter_2_num[move[2]]
  to_ouptut_layer[to_row,to_column] = 1
  return np.stack([from_output_layer, to_ouptut_layer])

def create_move_list(s):
  return re.sub('\d*\. ', '',s).split(' ')[:-1]
```

```
#Kaggle Dataset
chess_data_raw = pd.read_csv('/content/chess_games.csv',
   usecols =['AN','WhiteElo']) #asses only the columns we need
chess_data = chess_data_raw[chess_data_raw['WhiteElo']> 2000]
   #filter by elo
del chess_data_raw #delete games which are less then required
   elo
gc.collect() #clear RAMSPACE
chess_data = chess_data[['AN']] # del 'WhiteElo'
chess_data = chess_data[~chess_data['AN'].str.contains('{')]
   #clear up characters which code can't handle
chess_data = chess_data[chess_data['AN'].str.len()>20] #clear
   too short games
print(chess_data.shape[0])
```

Listing 1: The preprocess.py script used for preparing the data.

A.2 CNN Configuration Script

```
from preprocessing import create_move_list, board_2_rep,
   move_2_rep, chess_data
import numpy as np
# PyTorch Imports
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import chess

class ChessDataset(Dataset):
  def __init__(self,games):
```

```
    super(ChessDataset, self).__init__()
    self.games = games

  def __len__(self):
      #use 40k games
      return 40_000

  def __getitem__(self, index):
    game_i = np.random.randint(self.games.shape[0])
    random_game = chess_data['AN'].values[game_i]
    moves = create_move_list(random_game)
    game_state_i = np.random.randint(len(moves)-1)
    next_move = moves[game_state_i]
    moves = moves[:game_state_i]
    board = chess.Board()
    for move in moves :
      board.push_san(move)
    #feature map for played move
    x = board_2_rep(board)
    y = move_2_rep(next_move, board)
    if game_state_i % 2 == 1:
      x*= -1
    return x, y

#Network = several modules one after another
class Module(nn.Module):
    def __init__(self, hidden_size):
        super(Module, self).__init__()
        #2 Convolutional Layers
        self.conv1 = nn.Conv2d(hidden_size, hidden_size, 3,
           stride=1, padding=1)
```

```
        self.conv2 = nn.Conv2d(hidden_size, hidden_size, 3,
            stride=1, padding=1)
        #2BN layers
        self.bn1 = nn.BatchNorm2d(hidden_size)
        self.bn2 = nn.BatchNorm2d(hidden_size)
        #2BN layers
        self.activation1 = nn.SELU()
        self.activation2 = nn.SELU()

    def forward(self, x):
        x_input = torch.clone(x)
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.activation1(x)
        x = self.conv2(x)
        x = self.bn2(x)
        #Residual Connection
        x = x + x_input
        x = self.activation2(x)
        return x

class ChessNet(nn.Module):
    def __init__(self, hidden_layers=4, hidden_size=200):
        super(ChessNet, self).__init__()
        self.hidden_layers = hidden_layers
        self.input_layer = nn.Conv2d(6, hidden_size, 3,
            stride=1, padding=1)
        self.module_list = nn.ModuleList([Module(hidden_size)
            for i in range(hidden_layers)])
        self.output_layer = nn.Conv2d(hidden_size, 2, 3,
            stride=1, padding=1)
```

```
    def forward(self, x):
        x = self.input_layer(x)
        x = F.relu(x)
        for i in range(self.hidden_layers):
            x = self.module_list[i](x)
        x = self.output_layer(x)
        return x

# Dataloader
data_train = ChessDataset(chess_data['AN']) #training data with
   moves
data_train_loader = DataLoader(data_train, batch_size = 32,
   shuffle = True, drop_last=True)
```

Listing 2: The CNN_configuration.py script used for creating the Neural Netowrk.

A.3 My Personal GitHub

References

[1] Avneet Pannu. "Artificial intelligence and its application in different areas". In: *Artificial Intelligence* 4.10 (2015), pp. 79–84. URL: https://api.semanticscholar.org/CorpusID:113451322.

[2] David Kuntz and Angela K. Wilson. "Machine learning, artificial intelligence, and chemistry: How smart algorithms are reshaping simulation and the laboratory". In: *Pure and Applied Chemistry* 94.8 (2022). Accessed: 2024-12-14, pp. 1019–1054. DOI: doi:10.1515/pac-2022-0202. URL: https://doi.org/10.1515/pac-2022-0202.

[3] Jürgen Schmidhuber. "Deep learning in neural networks: An overview". In: *Neural networks : the official journal of the International Neural Network Society* 61 (2014), pp. 85–117. URL: https://api.semanticscholar.org/CorpusID:11715509.

[4] 3Blue1Brown. *But what is a neural network? | Deep learning chapter 1*. Accessed: 2024-10-15. 2017. URL: https://www.youtube.com/watch?v=aircAruvnKk&t=695s.

[5] Jianxin Wu. "Introduction to convolutional neural networks". In: *National Key Lab for Novel Software Technology. Nanjing University. China* 5 (2017), p. 495. URL: https://api.semanticscholar.org/CorpusID:36074296.

[6] Reid McIlroy-Young et al. "Aligning superhuman ai with human behavior: Chess as a model system". In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. Accessed: 2024-12-05. 2020, pp. 1677–1687. URL: https://www.maiachess.com/.

[7] P. Shanmugavadivu et al. "Bio-Optimization of Deep Learning Network Architectures". In: *Security and Communication Networks* 2022 (Sept. 2022). Accessed: 2024-11-25, pp. 1–11. DOI: 10.1155/2022/3718340.

[8] IBM Technology. *Machine Learning vs Deep Learning*. Accessed: 2024-11-21. 2022. URL: https://www.youtube.com/watch?v=q6kJ71tEYqM.

[9] Chaim Baskin et al. "Streaming Architecture for Large-Scale Quantized Neural Networks on an FPGA-Based Dataflow Platform". In: (July 2017). Accessed: 2024-12-25. DOI: 10.48550/arXiv.1708.00052.

[10] Sergey Ioffe and Christian Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *ArXiv* abs/1502.03167 (2015). URL: https://api.semanticscholar.org/CorpusID:5808102.

[11] Bakary Badjie. *Understanding the Intuition Behind Batch Normalization in CNNs*. Accessed: 2024-12-11. 2023. URL: https://medium.com/@bakarykumba1996/understanding-the-intuition-behind-batchnormalization-in-cnns-d4c3f43d2d37.

[12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet classification with deep convolutional neural networks". In: *Communications of the ACM* 60 (2012), pp. 84–90. URL: https://api.semanticscholar.org/CorpusID:195908774.

List of Figures

1 Hierarchy of AI subfields [2] 3

2 Human vs. AI chess strategy [6] 5

3 Feature Map of Pawns 6

4 Chess Board Representation from Python Chess Module 7

5 Basic Idea for Move Representation 8

6 Move Representation 9

7 The Raw Data from the Dataset 10

8 The Code for the Training Mechanism 12

9 Structure of a Simple NN vs CNN [7] 13

10 Functionality of a Kernel [9] 14

11 Batch-Normalization Layers [11] 15

12 Data Loader 16

13 The Output of the Network after an Epoch 19

14 Calculation of the Losses for both Matrices 20

15 Evaluation of a trained NN (Learning Rate = 0.01, 200 Epochs) 22

16 Evaluation of a Trained NN (Learning Rate = 0.005, 400 Epochs) . . . 23

Glossary

AI Artificial Intelligence. 3

BN Batch Normalization. 15

CNN Convolutional Neural Network. 4

CSV Comma-seperated values. 9

DL Deep Learning. 3, 14

GUI Graphical User Interface. 20

ML Machine Learning. 14

NN Neural Network. 3

RELU Rectified Linear Unit. 15

SAN Standard Algebraic Notation. 11

SELU Scaled Exponential Linear Unit. 15